

APOSTILA DO ALUNO — AULA 2

Requisito Funcional x Não-Funcional e Técnicas de Análise de Requisitos

Curso: Técnico em Desenvolvimento de Sistemas | Turma: 103513 | Disciplina: Modelagem de Sistemas (82h + 04h planejamento)

Data: 04/09/2026 (sexta-feira) | Horário: 07:30 às 11:30 (4h)

Objetivos desta aula

- Diferenciar requisito funcional (RF) de requisito não funcional (RNF).
- Conhecer técnicas de análise de requisitos.
- Transformar regras de negócio em requisitos funcionais e propor requisitos não funcionais.
- Reconhecer requisitos ambíguos ou conflitantes e propor correções.
- Aplicar a técnica MoSCoW para priorizar requisitos.

1. Retomando a Aula 1: as regras de negócio da TechBurger

Estas são as 8 regras de negócio identificadas no cenário TechBurger, já decompostas na Aula 1:

#	Regra (resumo)
1	Valor mínimo no aplicativo
2	Programa de fidelidade
3	Envio à cozinha após pagamento
4	Desconto PIX no app
5	Raio de entrega
6	Substituição de item em falta
7	Confirmação de pedidos grandes
8	Bloqueio por cancelamentos

2. Requisito Funcional (RF)

Um requisito funcional descreve o que o sistema deve fazer. É a aplicação prática de uma regra de negócio (ou de uma necessidade do usuário) em uma capacidade do sistema.

Um bom RF é redigido no padrão “o sistema deve + verbo de ação”, é objetivo e verificável — dá para testar se foi ou não atendido.

Exemplo:

Regra de negócio	Requisito Funcional
Pedido só é enviado à cozinha após confirmação de pagamento.	O sistema deve bloquear o envio do pedido para a cozinha até que o pagamento seja confirmado.

2.1 Como escrever um bom requisito funcional

Além do padrão “o sistema deve...”, um bom RF precisa ser:

- Claro e objetivo — uma frase, sem duplo sentido.
- Atômico — descreve uma única capacidade, não várias amarradas com “e”.
- Verificável — dá para escrever um teste que aprova ou reprovava o requisito.
- Rastreável — sempre é possível apontar de qual regra de negócio ele veio.

Compare:

Versão ruim	Por que é ruim	Versão melhorada
“O sistema deve gerenciar bem os pedidos.”	Vago: “gerenciar bem” não é verificável nem testável.	“O sistema deve permitir que o atendente altere um pedido enquanto ele ainda não foi confirmado.”

3. Requisito Não-Funcional (RNF)

Um requisito não funcional descreve como o sistema se comporta — não uma funcionalidade, mas uma qualidade do sistema.

Categoria	Exemplo (TechBurger)
Desempenho	O sistema deve confirmar o pagamento em até 3 segundos após o envio pelo cliente.
Segurança	Os dados de cartão e PIX devem ser criptografados em trânsito e em repouso.
Usabilidade	O aplicativo deve permitir finalizar um pedido simples em no máximo 4 toques na tela.
Disponibilidade	O sistema deve estar disponível 99,5% do tempo durante o horário de funcionamento da loja.
Confiabilidade	O sistema não deve permitir a perda de um pedido já pago, mesmo em caso de queda de conexão momentânea.
Manutenibilidade	Um desenvolvedor novo deve conseguir corrigir um bug simples no módulo de pedidos em até 1 dia de trabalho.
Portabilidade	O aplicativo deve funcionar tanto em Android quanto em iOS sem alterar as regras de negócio.
Escalabilidade	O sistema deve suportar 3 vezes o volume atual de pedidos sem queda de desempenho, em datas de alta demanda.

3.1 Pegadinhas comuns entre RF e RNF

Um teste rápido para não confundir os dois: se a frase descreve uma ação que o sistema executa, é RF; se descreve uma qualidade de como essa ação acontece, é RNF.

Frase	RF ou RNF?	Por quê
“O sistema deve permitir login	RF	Descreve uma funcionalidade (o

com e-mail e senha.”		que o sistema faz).
“O login deve responder em até 2 segundos.”	RNF (Desempenho)	Descreve uma qualidade do comportamento, não uma nova função.
“Senhas devem ser armazenadas de forma criptografada.”	RNF (Segurança)	Não é uma ação visível ao usuário, é uma exigência de qualidade.

3.2 Quiz relâmpago: RF ou RNF?

Agora que vocês já viram Requisito Funcional e Requisito Não-Funcional, testem o que aprenderam. Para cada frase, respondam: RF ou RNF? (respostas no Gabarito)

1. “O sistema deve permitir que o cliente cancele um pedido em até 5 minutos após a confirmação.”
2. “O cancelamento deve ser processado em até 1 segundo.”
3. “O sistema deve gerar um relatório mensal de vendas.”
4. “O relatório mensal deve poder ser exportado em PDF ou Excel.”
5. “O sistema deve funcionar corretamente mesmo com 500 pedidos simultâneos.”

4. Ambiguidade e Conflitos entre Requisitos

Mesmo seguindo o padrão “o sistema deve...”, um requisito ainda pode estar mal escrito. Dois problemas aparecem com frequência: ambiguidade e conflito entre requisitos.

4.1 Requisitos ambíguos

Um requisito é ambíguo quando admite mais de uma interpretação. As causas mais comuns são adjetivos vagos (“rápido”, “bom”, “fácil”), pronomes sem referência clara e o uso de “e/ou” sem definir prioridade.

Exemplo: “O sistema deve tratar bem os pedidos urgentes.” — Qual ação, exatamente? O que torna um pedido “urgente”? Sem responder isso, ninguém sabe testar o requisito.

4.2 Conflitos entre requisitos

Dois requisitos podem estar, cada um, bem escritos e ainda assim conflitarem entre si — atender totalmente um pode dificultar o outro.

Exemplo TechBurger: a regra “pedidos com mais de 10 itens exigem confirmação manual de um gerente” (controle/qualidade) pode entrar em tensão com um requisito futuro de desempenho como “todo pedido deve ser confirmado em até 1 minuto” — a confirmação manual pode não caber nesse tempo.

Como resolver: não ignorar o conflito — negociar a prioridade com o dono do negócio e documentar a decisão tomada, para que a equipe de desenvolvimento saiba qual requisito prevalece.

4.3 Pratique: ache o problema

Para cada requisito abaixo, identifiquem se o problema é ambiguidade, conflito, ou os dois — e proponham uma versão corrigida.

Requisito	Problema	Versão corrigida
“O sistema deve processar pedidos rapidamente.”		
“O aplicativo deve ser fácil de usar e seguro ao mesmo tempo, sem exigir login.”		
“Todo pedido deve ser entregue em até 20 minutos.” (considerando a regra de confirmação manual do gerente para pedidos grandes)		

5. Técnicas de análise de requisitos

5.1 Entrevista

Conversa direta e guiada com o stakeholder. A técnica mais usada para entender regras de negócio em profundidade.

Quando usar: poucos stakeholders, tema exige profundidade — mas consome mais tempo por pessoa ouvida.

5.2 Questionário

Perguntas padronizadas aplicadas a muitas pessoas ao mesmo tempo — útil quando há muitos usuários a ouvir.

Quando usar: muitos usuários a ouvir rapidamente, mas pouca profundidade em cada resposta.

5.3 Observação

Acompanhar o usuário executando o trabalho real, sem interferir — revela detalhes que a pessoa nem lembraria de contar numa entrevista.

Quando usar: comportamento real difere do que a pessoa relata verbalmente sobre o próprio trabalho.

5.4 Brainstorming

Geração coletiva de ideias em grupo, útil nas fases iniciais de um projeto.

Quando usar: fase inicial, poucas restrições ainda definidas — não serve para validar detalhes técnicos.

5.5 Prototipação

Construção de uma versão simples (tela, fluxo) para validar o entendimento com o usuário antes de desenvolver de verdade.

Quando usar: já existe uma ideia de tela ou fluxo e é preciso confirmar com o usuário antes de programar de verdade.

5.6 Análise de documentos

Estudo de manuais, contratos, planilhas e sistemas legados já existentes na empresa. Útil para regularizar regras que já estão em vigor, mas nunca foram formalizadas em nenhum lugar.

5.7 Workshop de requisitos (JAD)

Joint Application Development: reunião estruturada com vários stakeholders ao mesmo tempo, mediada por um facilitador, para levantar e já validar requisitos em conjunto — reduz idas e vindas entre entrevistas separadas.

5.8 Qual técnica usar em cada situação?

Situação	Técnica mais indicada
Poucos stakeholders, tema complexo e sensível	Entrevista
Centenas de usuários finais espalhados	Questionário
O usuário não sabe explicar o que faz, só faz	Observação
Início de projeto, ainda sem direção clara	Brainstorming
Validar uma tela ou fluxo antes de programar	Prototipação
Regras já existem, mas não estão documentadas	Análise de documentos
Muitos decisores, prazo curto para alinhar tudo	Workshop (JAD)

6. Priorização de Requisitos (MoSCoW)

Depois de levantar e escrever bons requisitos, ainda falta decidir o que entra primeiro no sistema — principalmente quando o tempo ou o orçamento são limitados. O método MoSCoW ajuda nessa priorização, classificando cada requisito em uma de quatro categorias:

- Must have — essencial; sem ele o sistema não cumpre seu propósito.
- Should have — importante, mas o sistema ainda funciona sem ele no lançamento.
- Could have — desejável, entra apenas se sobrar tempo ou recurso.
- Won't have (por agora) — fica fora do escopo atual, sem descartar para o futuro.

6.1 Exemplo aplicado ao TechBurger

Categoria	Requisito	Por quê
Must	“O sistema deve enviar o pedido para a cozinha somente após a confirmação do pagamento.”	Sem isso, a operação financeira e a cozinha ficam desalinhadas.
Should	“O sistema deve aplicar 5% de desconto automático quando o pagamento for via PIX.”	Importante comercialmente, mas o sistema roda sem isso.
Could	“O aplicativo deve sugerir um item substituto quando o item selecionado estiver em falta.”	Melhora a experiência, não é crítico para concluir o pedido.
Won't (por agora)	“O sistema deve recomendar produtos usando inteligência artificial.”	Fora do escopo desta primeira versão do sistema.

6.2 Por que priorizar importa

Priorizar ajuda o time e o cliente a alinharem expectativas sobre o que estará pronto na primeira entrega, e evita que funcionalidades “seria legal ter” atrasem o que é essencial para o negócio funcionar.

7. Atividade prática

7.1 Role-play de entrevista

- Em duplas: um é o “dono do negócio TechBurger”, outro é o “analista de requisitos”.
- O analista faz perguntas abertas para tentar redescobrir 2-3 regras de negócio, sem consultar o material.
- Troquem os papéis na segunda metade do tempo.

7.2 Das regras aos requisitos

Em grupo, transformem cada uma das 8 regras de negócio em um Requisito Funcional (padrão “o sistema deve...”) e proponham pelo menos 4 Requisitos Não-Funcionais com sua categoria.

Exemplo já resolvido, para servir de modelo (a regra “Envio à cozinha após pagamento” já está pronta na tabela abaixo — não repita essa linha, comece pela regra 1):

Regra (resumo)	Requisito Funcional
Envio à cozinha após pagamento	O sistema deve liberar o pedido para a tela da cozinha somente após a confirmação do pagamento.

Regra (resumo)	Requisito Funcional
Valor mínimo no aplicativo	
Programa de fidelidade	
Envio à cozinha após pagamento	
Desconto PIX no app	
Raio de entrega	
Substituição de item em falta	
Confirmação de pedidos grandes	
Bloqueio por cancelamentos	

Categoria do RNF	Requisito Não-Funcional proposto

7.3 Priorização MoSCoW dos requisitos

Peguem os Requisitos Funcionais que o grupo escreveu na atividade 7.2 e classifiquem cada um em Must, Should, Could ou Won't have (seção 6), justificando pelo menos 2 escolhas.

Requisito (resumo)	Categoria MoSCoW	Justificativa

8. Critérios de Aceite (material de aprofundamento)

Um critério de aceite é uma condição objetiva que diz quando um Requisito Funcional pode ser considerado “pronto e correto”. Ele fecha o ciclo regra de negócio → requisito → critério de aceite → teste, que será retomado nas aulas de teste de sistemas mais adiante no curso.

8.1 Formato simples: Dado / Quando / Então

Dado [um contexto inicial], Quando [uma ação acontece], Então [o sistema deve reagir assim].

Exemplo, para o RF “o sistema deve bloquear o envio do pedido para a cozinha até que o pagamento seja confirmado”:

- Dado um pedido com pagamento pendente, Quando o atendente tentar enviar o pedido à cozinha, Então o sistema deve impedir o envio e exibir uma mensagem de pagamento pendente.

8.2 Pratique

Escolham mais 1 RF da tabela da atividade 7.2 e escrevam o critério de aceite dele, no formato Dado/Quando/Então.

9. Material extra: aplicando tudo à Biblioteca “Ler Mais” (se sobrar tempo)

Lembram do exemplo da Biblioteca “Ler Mais”, da Aula 1? Vamos aplicar o mesmo processo de hoje — RF, RNF, ambiguidade/conflito e MoSCoW — a esse outro cenário, para treinar o olho em um domínio diferente do TechBurger.

Regra: associado com 3 livros emprestados não pode pegar um 4º; professor tem prazo de 30 dias em vez de 14; devolução atrasada gera multa de R\$ 1,00/dia, exceto livro de leitura escolar.

Artefato	Exemplo resolvido
RF	O sistema deve recusar um novo empréstimo quando o associado já tiver 3 livros emprestados ao mesmo tempo.
RF	O sistema deve aplicar automaticamente o prazo de 30 dias quando o associado tiver o perfil “professor”.
RNF (Usabilidade)	O atendente deve conseguir registrar um novo

	empréstimo em no máximo 3 telas.
Ambíguo? corrija	“O sistema deve avisar o associado rapidamente sobre a multa.” → “O sistema deve notificar o associado por e-mail em até 1 hora após a devolução em atraso.”
MoSCoW	Recusar o 4º empréstimo = Must (regra central do negócio); notificar por e-mail = Should (importante, mas o sistema funciona sem isso no dia 1).

10. Erros comuns em entrevista de requisitos (se sobrar tempo)

Voltando ao role-play do Bloco 7: aqui estão os erros mais comuns de quem está aprendendo a levantar requisitos por entrevista.

- Perguntas fechadas demais (sim/não) em vez de abertas — perde detalhes importantes que o entrevistado nem lembraria de contar.
- Induzir a resposta (“você não acha que seria melhor...?”) em vez de deixar o stakeholder responder livremente.
- Não validar o que ouviu — não repetir/resumir de volta para o entrevistado, para confirmar que o entendimento está correto.
- Focar em “como” o sistema deve funcionar em vez de “o que” o negócio precisa — viés de solução prematura, antes de entender o problema.
- Não registrar a conversa e confiar só na memória depois — detalhes se perdem entre a entrevista e a redação dos requisitos.

Reflexão: vocês sentiram algum desses erros durante o role-play do Bloco 7? Qual, e o que fariam diferente?

11. Síntese: fechando o ciclo

Tudo que fizemos hoje encaixa numa única cadeia, do negócio até o código:

Regra de negócio → Requisito (RF ou RNF) → checar Ambiguidade/Conflito → Priorizar (MoSCoW) → Critério de Aceite → (Aula 3: representar tudo em UML)

Cada seta é uma pergunta que vocês já sabem responder: o que o negócio precisa? isso vira o quê no sistema? está claro e sem conflito? entra na primeira versão? como sei que ficou pronto?

Glossário

Termo	Significado
Requisito Funcional (RF)	Descreve o que o sistema deve fazer.
Requisito Não-Funcional (RNF)	Descreve como o sistema deve se comportar (qualidade).
Stakeholder	Pessoa ou grupo interessado ou afetado pelo sistema.

Prototipação	Construção de uma versão simplificada para validar entendimento.
Verificável	Característica de um requisito que pode ser testada/comprovada.
Manutenibilidade	Facilidade de corrigir, adaptar ou evoluir o sistema depois de pronto.
Portabilidade	Capacidade de o sistema funcionar em diferentes ambientes/plataformas sem grandes mudanças.
Análise de documentos	Técnica de levantar requisitos a partir de manuais, contratos e sistemas já existentes.
JAD (Joint Application Development)	Workshop estruturado com vários stakeholders para levantar e validar requisitos em conjunto.
Ambiguidade	Quando um requisito admite mais de uma interpretação possível.
Conflito de requisitos	Quando dois requisitos, mesmo bem escritos, competem entre si e não podem ser plenamente satisfeitos ao mesmo tempo.
MoSCoW	Técnica de priorização de requisitos em quatro categorias: Must have, Should have, Could have e Won't have.
Critério de aceite	Condição objetiva (formato Dado/Quando/Então) que define quando um requisito pode ser considerado pronto e correto.

Perguntas de revisão

1. Qual a diferença entre requisito funcional e requisito não funcional?
2. Cite três técnicas de análise de requisitos e uma situação em que cada uma seria mais indicada.
3. Reescreva a regra “clientes cadastrados no programa de fidelidade acumulam 1 ponto a cada R\$10,00” como um Requisito Funcional.
4. Dê um exemplo de Requisito Não-Funcional de segurança para um sistema de pagamentos.
5. Por que um requisito precisa ser “verificável”?
6. Classifique cada frase como RF ou RNF: (a) “O sistema deve emitir a nota fiscal do pedido”; (b) “O sistema deve emitir a nota fiscal em até 5 segundos”.
7. Cite uma situação em que o Workshop (JAD) é mais indicado do que uma sequência de entrevistas individuais, e explique por quê.
8. Dê um exemplo de Requisito Não-Funcional de manutenibilidade e um de portabilidade para o sistema da TechBurger.
9. Reescreva o RF ruim “O sistema deve gerenciar bem os pedidos” seguindo os critérios da seção 2.1.
10. O que torna um requisito ambíguo? Dê um exemplo (pode ser um dos que você mesmo escreveu na atividade) e reescreva-o de forma mais clara.
11. Explique a diferença entre um requisito “Must have” e um “Could have” no método MoSCoW, usando um exemplo do cenário TechBurger.
12. Escreva o critério de aceite (Dado/Quando/Então) para o RF “o sistema deve exigir confirmação manual de um gerente para pedidos com mais de 10 itens”.
13. Um grupo escreveu o RNF “O sistema deve ser seguro.” Explique por que esse RNF está ambíguo e reescreva-o classificando também sua categoria MoSCoW.